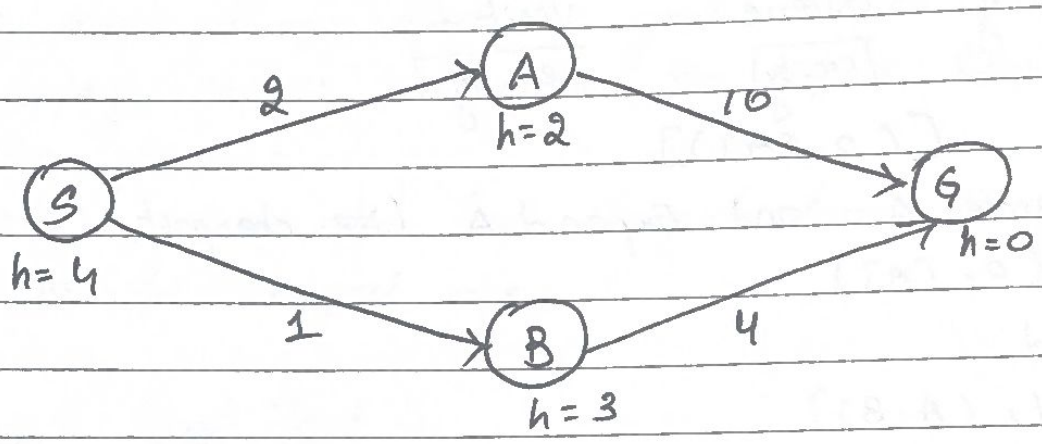
Lab-4 Greedy Best First Search

To implement the Greedy Best First Search algorithm and find a path from the start node to the goal node using the heuristic value (h).



Step 1: Creating the Graph

```
graph = {'S': [('A', 2), ('B', 1)],
         'A' : [('G', 10)],
         'B' : [('G', 4)],
         'G' : []}
```

The graph is created using a dictionary. It holds the path and how much it costs to reach there.

Step 2: Creating Heuristic Dictionary

```
h = {'S': 4, 'A': 2, 'B': 3, 'G': 0}
```

h holds the heuristic values of each node i.e. the destimated distance needed to reach the goal state G.
The heuristic should never overestimate

Step 3: Calculating Actual Cost

```
def cost_of(path):
    return sum([w for n, w in graph[a] if n == b][0]
               for a, b in zip(path, path[1:]))
```

zip() pairs consecutive nodes: ['S', 'A', 'G'] → ('S', 'A'), ('S', 'G')
[0] gets the actual number from one item list

classmate
Date
Page

Step 4: Declare the queue

```
def greedy (start, goal):
    queue = [(h [start] , [start])]
```

the queue is sorted by 'h' i.e. heuristic value.

Step 5: Sort and pop

```
while queue:
    queue.sort ()
    _, path = queue.pop(0)
    node = path [-1]
```

The _, path means that in python the value is unpacked but is not needed. We only need the path and not the heuristic value once it has been calculated.

Step 6: Goal Test

```
if node == goal:
    return path, cost_of (path)
```

In greedy we dont calculate the cost while searching for a path. When we find the path at last then we calculate the cost of the path

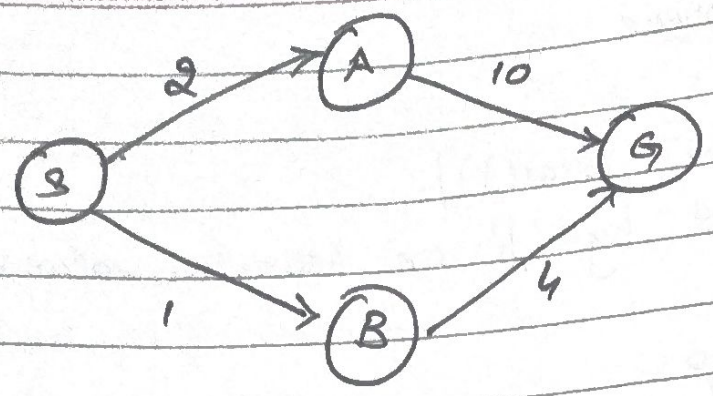
Step 7: Expand

```
for n,w in graph [node]:
    queue.append ((h[n], path + [n]))
```

Gives the neighbour nodes & the the real edge cost.

Step 8: Call & Print

```
path, cost = greedy ('S', 'G')
print ("Greedy path :", path)
print ("Actual cost :", cost)
```



$h(S) = 4$, $h(A) = 2$, $h(B) = 3$, $h(G) = 0$

Add S (4, S)

S has two neighbours

Remove S

Add A, B

(2, [S, A])

(3, [S, B])

Sort based on heuristic

Choose A

current path $S \rightarrow A$

A has one neighbour G

Add G

(0, [S, A, G])

(3, [S, B])

Smallest is 0 so

current path $S \rightarrow A \rightarrow G$

Goal node reache

Actual cost → $S \rightarrow A = 2$ +

$A \rightarrow G = 10$

$= 12$